

1. Download C-MAPSS dataset <https://data.nasa.gov/dataset/cmapss-jet-engine-simulated-data>. The CMAPSS dataset has vibration, temperature, pressures, flow rate and operating conditions of a turbofan engine.
2. Understanding the data by parsing text to pandas data frames and label RUL by creating a target column.
3. Store these as csv/excel files in a folder. Visualize the degradation trends in the CMAPSS dataset. You should normalize the data using a suitable normalizer and remove non informative data columns.
4. Prepare a short writeup of which columns you are removing and why.
5. Generate sliding windows as time-series sequences and keep them in a separate files in separate folders.
6. For all the above compute rolling statistics, trend feature like slope monotonicity, health index creation and use PCA to decide which columns to drop. [check for sensor1, sensor5, sensor6, sensor10, sensor16, sensor18, sensor19]

Use FD001 -> simple (1 operating condition, low noise FD004 -> complex (multiple conditions, high noise)

Weeks 1–2 → FD001 Weeks 3–4 → FD004 + transfer tests
 $RUL = \max_cycle - \text{current_cycle}$
 $RUL = \min(RUL, 125)$ This is a standard practice in literature
 window_size = 30 features ≈ 14–18 Loss Function :

MSELoss (primary)

7. Implement a three layer neural network using Input vector (~420–540 features), Architecture : Dense(128) → ReLU, Dense(64) → ReLU, Dense(1) [output layer]
 8. Present your RMSE results.
- Keep all data files, all implementations on neural networks will be using the processed C-MAPSS datasets

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
```

Defining Column Names & Load Data

```
In [3]: # C-MAPSS has 26 columns: unit, cycle, 3 op settings, 21 sensors

col_names = ['unit_id', 'cycle',
             'op_setting1', 'op_setting2', 'op_setting3',
             'sensor1', 'sensor2', 'sensor3', 'sensor4', 'sensor5',
             'sensor6', 'sensor7', 'sensor8', 'sensor9', 'sensor10',
```

```

        'sensor11', 'sensor12', 'sensor13', 'sensor14', 'sensor15',
        'sensor16', 'sensor17', 'sensor18', 'sensor19', 'sensor20', 'sens

def load_cmapss(file_path):
    df = pd.read_csv(file_path, sep='\s+', header=None, names=col_names)
    return df

# Load FD001 train and test
train_df = load_cmapss('C-MAPSS/CMAPSSData/train_FD001.txt')
test_df = load_cmapss('C-MAPSS/CMAPSSData/test_FD001.txt')
rul_df = pd.read_csv('C-MAPSS/CMAPSSData/RUL_FD001.txt', header=None, n

print(train_df.shape)
train_df.head()

```

(20631, 26)

Out[3]:

	unit_id	cycle	op_setting1	op_setting2	op_setting3	sensor1	sensor2	sensor3	se
0	1	1	-0.0007	-0.0004	100.0	518.67	641.82	1589.70	1.
1	1	2	0.0019	-0.0003	100.0	518.67	642.15	1591.82	1.
2	1	3	-0.0043	0.0003	100.0	518.67	642.35	1587.99	1.
3	1	4	0.0007	0.0000	100.0	518.67	642.35	1582.79	1.
4	1	5	-0.0019	-0.0002	100.0	518.67	642.37	1582.85	1.

5 rows × 26 columns



Adding RUL column to training data

In [4]:

```

# RUL = max_cycle for that engine - current cycle
def add_rul(df):
    max_cycles = df.groupby('unit_id')['cycle'].max().reset_index()
    max_cycles.columns = ['unit_id', 'max_cycle']
    df = df.merge(max_cycles, on='unit_id')
    df['RUL'] = df['max_cycle'] - df['cycle']
    df['RUL'] = df['RUL'].clip(upper=125) # cap at 125
    df.drop(columns=['max_cycle'], inplace=True)
    return df

train_df = add_rul(train_df)
print(train_df[['unit_id', 'cycle', 'RUL']].head(10))

```

	unit_id	cycle	RUL
0	1	1	125
1	1	2	125
2	1	3	125
3	1	4	125
4	1	5	125
5	1	6	125
6	1	7	125
7	1	8	125
8	1	9	125
9	1	10	125

Adding RUL to Test Data (using the provided RUL file)

```
In [5]: # For test set, the last cycle's RUL is given in RUL_FD001.txt

def add_rul_test(df, rul_df):
    # rul_df has one RUL value per engine (the RUL at the LAST recorded cycle)
    last_cycles = df.groupby('unit_id')['cycle'].max().reset_index()
    last_cycles.columns = ['unit_id', 'last_cycle']
    last_cycles['true_rul_at_end'] = rul_df['RUL'].values

    df = df.merge(last_cycles, on='unit_id')
    df['RUL'] = (df['last_cycle'] - df['cycle']) + df['true_rul_at_end']
    df['RUL'] = df['RUL'].clip(upper=125)
    df.drop(columns=['last_cycle', 'true_rul_at_end'], inplace=True)
    return df

test_df = add_rul_test(test_df, rul_df)
test_df[['unit_id', 'cycle', 'RUL']].head(10)
```

Out[5]:

	unit_id	cycle	RUL
0	1	1	125
1	1	2	125
2	1	3	125
3	1	4	125
4	1	5	125
5	1	6	125
6	1	7	125
7	1	8	125
8	1	9	125
9	1	10	125

Saving data to CSV

```
In [6]: os.makedirs('C-MAPSS/CMAPSSData/processed', exist_ok=True)

train_df.to_csv('C-MAPSS/CMAPSSData/processed/train_FD001.csv', index=False)
test_df.to_csv('C-MAPSS/CMAPSSData/processed/test_FD001.csv', index=False)

print("Saved!")
print(f"Train shape: {train_df.shape}")
print(f"Test shape: {test_df.shape}")
```

Saved!

Train shape: (20631, 27)

Test shape: (13096, 27)

Visualize Sensor Degradation Trends

```
In [7]: # Plot a few sensors for a single engine to see degradation
sensor_cols = [f'sensor{i}' for i in range(1, 22)]
```

```

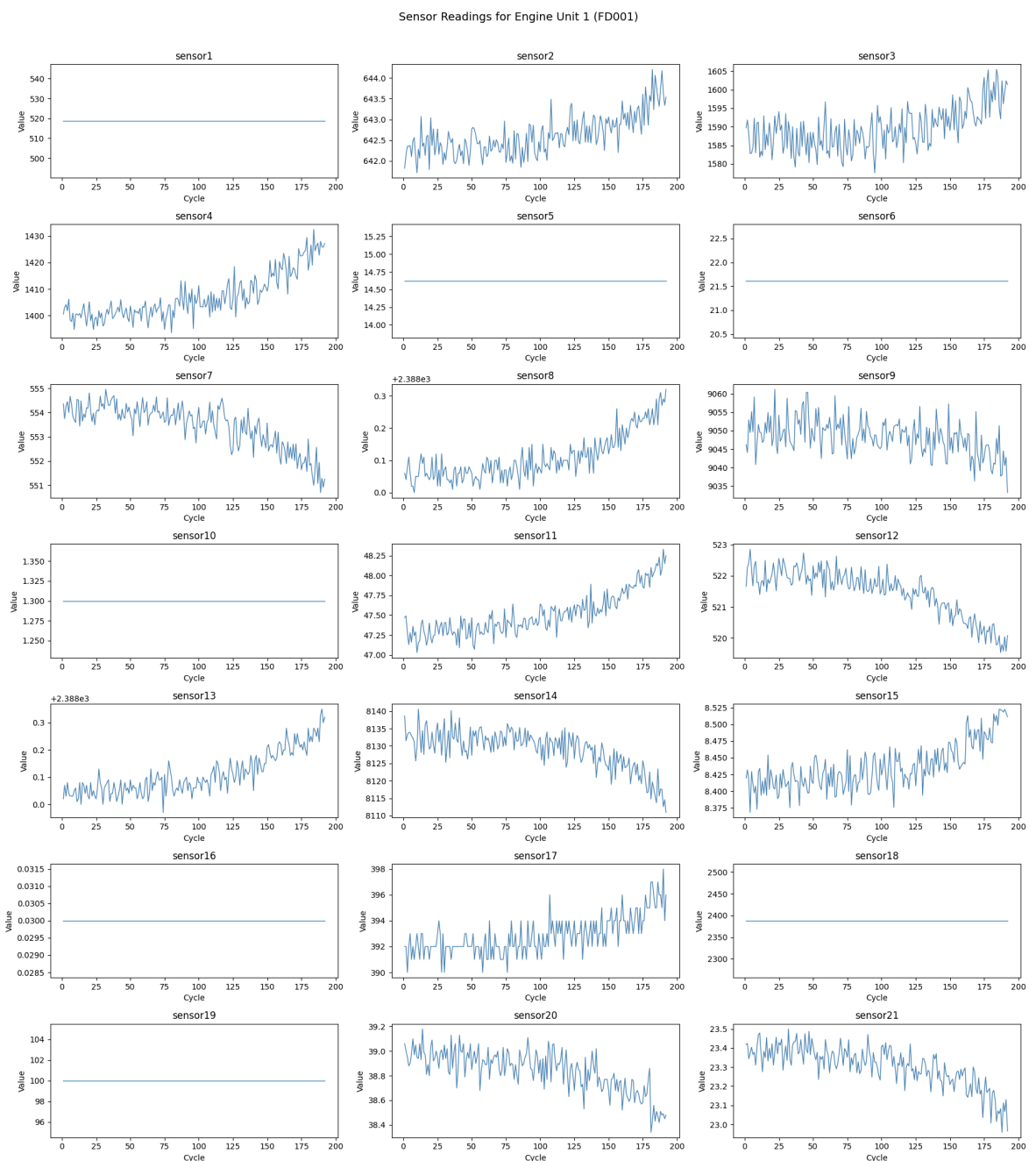
fig, axes = plt.subplots(7, 3, figsize=(18, 20))
axes = axes.flatten()

unit1 = train_df[train_df['unit_id'] == 1].reset_index(drop=True)

for i, sensor in enumerate(sensor_cols):
    axes[i].plot(unit1['cycle'], unit1[sensor], color='steelblue', linewidth=2)
    axes[i].set_title(sensor)
    axes[i].set_xlabel('Cycle')
    axes[i].set_ylabel('Value')

plt.suptitle('Sensor Readings for Engine Unit 1 (FD001)', fontsize=14, y=1.05)
plt.tight_layout()
plt.savefig('C-MAPSS/CMAPSSData/processed/sensor_trends.png', dpi=150, bbox_inches='tight')
plt.show()

```



```

In [8]: # Sensors with near-zero std across the whole dataset carry no information
sensor_std = train_df[sensor_cols].std()
print("Standard Deviation per Sensor:")
print(sensor_std.round(6))

```

```
# Flag sensors with very low std
low_std_sensors = sensor_std[sensor_std < 0.01].index.tolist()
print(f"\nLow-variance sensors (std < 0.01): {low_std_sensors}")
```

Standard Deviation per Sensor:

```
sensor1      0.000000
sensor2      0.500053
sensor3      6.131150
sensor4      9.000605
sensor5      0.000000
sensor6      0.001389
sensor7      0.885092
sensor8      0.070985
sensor9      22.082880
sensor10     0.000000
sensor11     0.267087
sensor12     0.737553
sensor13     0.071919
sensor14     19.076176
sensor15     0.037505
sensor16     0.000000
sensor17     1.548763
sensor18     0.000000
sensor19     0.000000
sensor20     0.180746
sensor21     0.108251
dtype: float64
```

Low-variance sensors (std < 0.01): ['sensor1', 'sensor5', 'sensor6', 'sensor10', 'sensor16', 'sensor18', 'sensor19']

dROPPING non-informative sensors

```
In [9]: sensors_to_drop = ['sensor1', 'sensor5', 'sensor6', 'sensor10', 'sensor16',
                           'sensor18', 'sensor19']

train_clean = train_df.drop(columns=sensors_to_drop)
test_clean = test_df.drop(columns=sensors_to_drop)

remaining_sensors = [c for c in train_clean.columns if 'sensor' in c]
print(f"Remaining sensors ({len(remaining_sensors)}): {remaining_sensors}")
print(f"Train shape after dropping: {train_clean.shape}")
```

Remaining sensors (14): ['sensor2', 'sensor3', 'sensor4', 'sensor7', 'sensor8', 'sensor9', 'sensor11', 'sensor12', 'sensor13', 'sensor14', 'sensor15', 'sensor17', 'sensor20', 'sensor21']
Train shape after dropping: (20631, 20)

Normalize the Sensor Columns

```
In [10]: from sklearn.preprocessing import MinMaxScaler

feature_cols = [c for c in train_clean.columns if c not in ['unit_id', 'c

scaler = MinMaxScaler()
train_clean[feature_cols] = scaler.fit_transform(train_clean[feature_cols])

# Use SAME scaler on test (fit only on train - important!)
test_clean[feature_cols] = scaler.transform(test_clean[feature_cols])
```

```
print("Normalized! Sample:")
train_clean[feature_cols].describe().round(3)
```

Normalized! Sample:

Out[10]:

	op_setting1	op_setting2	op_setting3	sensor2	sensor3	sensor4	sensor5
count	20631.000	20631.000	20631.0	20631.000	20631.000	20631.000	20631.000
mean	0.499	0.502	0.0	0.443	0.425	0.450	0.443
std	0.126	0.244	0.0	0.151	0.134	0.152	0.151
min	0.000	0.000	0.0	0.000	0.000	0.000	0.000
25%	0.414	0.333	0.0	0.336	0.332	0.339	0.333
50%	0.500	0.500	0.0	0.431	0.416	0.435	0.431
75%	0.586	0.750	0.0	0.539	0.509	0.545	0.539
max	1.000	1.000	0.0	1.000	1.000	1.000	1.000

```
In [11]: train_clean.to_csv('C-MAPSS/CMAPSSData/processed/train_FD001_clean.csv',
test_clean.to_csv('C-MAPSS/CMAPSSData/processed/test_FD001_clean.csv', index=False)

print(f"Clean train shape: {train_clean.shape}")
print(f"Clean test shape: {test_clean.shape}")
print(f"Feature columns: {feature_cols}")
```

Clean train shape: (20631, 20)

Clean test shape: (13096, 20)

Feature columns: ['op_setting1', 'op_setting2', 'op_setting3', 'sensor2', 'sensor3', 'sensor4', 'sensor5', 'sensor6', 'sensor7', 'sensor8', 'sensor9', 'sensor10', 'sensor11', 'sensor12', 'sensor13', 'sensor14', 'sensor15', 'sensor16', 'sensor17', 'sensor18', 'sensor19', 'sensor20']

Sliding Windows

```
In [12]: WINDOW_SIZE = 30

def create_windows(df, window_size, feature_cols):
    X, y = [], []

    for unit_id in df['unit_id'].unique():
        unit_data = df[df['unit_id'] == unit_id].reset_index(drop=True)

        # Need at least window_size rows to form one sample
        if len(unit_data) < window_size:
            continue

        for i in range(len(unit_data) - window_size + 1):
            window = unit_data[feature_cols].iloc[i : i + window_size].values
            rul = unit_data['RUL'].iloc[i + window_size - 1]
            X.append(window)
            y.append(rul)

    return np.array(X), np.array(y)

feature_cols = [c for c in train_clean.columns if c not in ['unit_id', 'RUL']]
```

```
X_train, y_train = create_windows(train_clean, WINDOW_SIZE, feature_cols)
X_test, y_test = create_windows(test_clean, WINDOW_SIZE, feature_cols)

print(f"X_train shape : {X_train.shape}") # (samples, 30, 17)
print(f"y_train shape : {y_train.shape}")
print(f"X_test shape : {X_test.shape}")
print(f"y_test shape : {y_test.shape}")
```

```
X_train shape : (17731, 30, 17)
y_train shape : (17731,)
X_test shape : (10196, 30, 17)
y_test shape : (10196,)
```

```
In [14]: # We use a Dense (fully connected) network for the assignment
# So we flatten (30, 17) → (510,) per sample

X_train_flat = X_train.reshape(X_train.shape[0], -1)
X_test_flat = X_test.reshape(X_test.shape[0], -1)

print(f"Flattened X_train : {X_train_flat.shape}") # (samples, 510)
print(f"Flattened X_test : {X_test_flat.shape}")
print(f"Input vector size : {X_train_flat.shape[1]} features ")
```

```
Flattened X_train : (17731, 510)
Flattened X_test : (10196, 510)
Input vector size : 510 features
```

Saving windows to separate folder

```
In [15]: import os

windows_dir = 'C-MAPSS/CMAPSSData/windows'
os.makedirs(windows_dir, exist_ok=True)

np.save(os.path.join(windows_dir, 'X_train.npy'), X_train_flat)
np.save(os.path.join(windows_dir, 'y_train.npy'), y_train)
np.save(os.path.join(windows_dir, 'X_test.npy'), X_test_flat)
np.save(os.path.join(windows_dir, 'y_test.npy'), y_test)

print("Saved windows!")
print(f"\nFolder contents:")
for f in os.listdir(windows_dir):
    size = os.path.getsize(os.path.join(windows_dir, f)) / 1024
    print(f" {f} → {size:.1f} KB")
```

Saved windows!

Folder contents:

```
X_test.npy → 40624.8 KB
y_test.npy → 79.8 KB
y_train.npy → 138.6 KB
X_train.npy → 70647.1 KB
```

NN

Loading windows

```
In [16]: import numpy as np

windows_dir = 'C-MAPSS/CMAPSSData/windows'

X_train = np.load(f'{windows_dir}/X_train.npy')
y_train = np.load(f'{windows_dir}/y_train.npy')
X_test = np.load(f'{windows_dir}/X_test.npy')
y_test = np.load(f'{windows_dir}/y_test.npy')

print(f"X_train : {X_train.shape}")
print(f"y_train : {y_train.shape}")
print(f"X_test : {X_test.shape}")
print(f"y_test : {y_test.shape}")
```

```
X_train : (17731, 510)
y_train : (17731,)
X_test : (10196, 510)
y_test : (10196,)
```

Model

```
In [17]: import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

print(f"TensorFlow version: {tf.__version__}")

input_dim = X_train.shape[1] # 510

def build_model(input_dim):
    model = keras.Sequential([
        # Layer 1
        layers.Dense(128, activation='relu', input_shape=(input_dim,)),
        layers.BatchNormalization(),
        layers.Dropout(0.2),

        # Layer 2
        layers.Dense(64, activation='relu', name='dense_2'),
        layers.BatchNormalization(),
        layers.Dropout(0.2),

        # Output layer
        layers.Dense(1, name='output') # RUL prediction (regression)
    ])
    return model

model = build_model(input_dim)
model.summary()
```



```
2026-04-10 15:38:58.349249: I tensorflow/core/util/port.cc:153] oneDNN cus
tom operations are on. You may see slightly different numerical results du
e to floating-point round-off errors from different computation orders. To
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-04-10 15:38:58.569231: I tensorflow/core/platform/cpu_feature_guard.c
c:210] This TensorFlow binary is optimized to use available CPU instructio
ns in performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operatio
ns, rebuild TensorFlow with the appropriate compiler flags.
2026-04-10 15:38:59.909328: I tensorflow/core/util/port.cc:153] oneDNN cus
tom operations are on. You may see slightly different numerical results du
e to floating-point round-off errors from different computation orders. To
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
TensorFlow version: 2.20.0
```

```
/home/saurabh-singh/miniconda3/envs/tf-gpu/lib/python3.10/site-packages/ke
ras/src/layers/core/dense.py:95: UserWarning: Do not pass an `input_shape
`/`input_dim` argument to a layer. When using Sequential models, prefer us
ing an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
WARNING: All log messages before absl::InitializeLog() is called are writt
en to STDERR
I0000 00:00:1775815740.535205 10391 gpu_device.cc:2020] Created device /
job:localhost/replica:0/task:0/device:GPU:0 with 10247 MB memory: -> devi
ce: 0, name: NVIDIA GeForce RTX 3060, pci bus id: 0000:01:00.0, compute ca
pability: 8.6
```

Model: "sequential"

Layer (type)	Output Shape	
dense_1 (Dense)	(None, 128)	
batch_normalization (BatchNormalization)	(None, 128)	
dropout (Dropout)	(None, 128)	
dense_2 (Dense)	(None, 64)	
batch_normalization_1 (BatchNormalization)	(None, 64)	
dropout_1 (Dropout)	(None, 64)	
output (Dense)	(None, 1)	

Total params: 74,497 (291.00 KB)

Trainable params: 74,113 (289.50 KB)

Non-trainable params: 384 (1.50 KB)

Compile and training

```
In [18]: model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss='mse',          # MSELoss as required by assignment
    metrics=['mae']
)
```

```

# Callbacks
callbacks = [
    keras.callbacks.EarlyStopping(
        monitor='val_loss', patience=10, restore_best_weights=True, verbose=1
    ),
    keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss', factor=0.5, patience=5, verbose=1
    ),
    keras.callbacks.ModelCheckpoint(
        filepath='../CMAPSSData/best_model.keras',
        monitor='val_loss', save_best_only=True, verbose=1
    )
]

history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=256,
    callbacks=callbacks,
    verbose=1
)

```

Epoch 1/100

```

2026-04-10 15:39:43.779495: I external/local_xla/xla/service/service.cc:163] XLA service 0x7d37980179b0 initialized for platform CUDA (this does not guarantee that XLA will be used). Devices:
2026-04-10 15:39:43.779512: I external/local_xla/xla/service/service.cc:171] StreamExecutor device (0): NVIDIA GeForce RTX 3060, Compute Capability 8.6
2026-04-10 15:39:43.810716: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:269] disabling MLIR crash reproducer, set env var `MLIR_CRASH_REPRODUCER_DIRECTORY` to enable.
2026-04-10 15:39:43.973129: I external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:473] Loaded cuDNN version 91701
2026-04-10 15:39:44.088372: I external/local_xla/xla/service/gpu/autotuning/dot_search_space.cc:208] All configs were filtered out because none of them sufficiently match the hints. Maybe the hints set does not contain a good representative set of valid configs? Working around this by using the full hints set instead.
2026-04-10 15:39:44.088418: I external/local_xla/xla/service/gpu/autotuning/dot_search_space.cc:208] All configs were filtered out because none of them sufficiently match the hints. Maybe the hints set does not contain a good representative set of valid configs? Working around this by using the full hints set instead.
2026-04-10 15:39:45.050068: I external/local_xla/xla/stream_executor/cuda/subprocess_compilation.cc:346] ptxas warning : Registers are spilled to local memory in function 'gemm_fusion_dot_1400', 124 bytes spill stores, 124 bytes spill loads

```

35/56 ————— 0s 1ms/step - loss: 7639.3650 - mae: 78.6423

```

I0000 00:00:1775815786.199661 35406 device_compiler.h:196] Compiled cluster using XLA! This line is logged at most once for the lifetime of the process.
2026-04-10 15:39:46.553001: I external/local_xla/xla/service/gpu/autotuning/dot_search_space.cc:208] All configs were filtered out because none of them sufficiently match the hints. Maybe the hints set does not contain a good representative set of valid configs? Working around this by using the full hints set instead.
2026-04-10 15:39:46.553049: I external/local_xla/xla/service/gpu/autotuning/dot_search_space.cc:208] All configs were filtered out because none of them sufficiently match the hints. Maybe the hints set does not contain a good representative set of valid configs? Working around this by using the full hints set instead.
2026-04-10 15:39:46.553093: I external/local_xla/xla/service/gpu/autotuning/dot_search_space.cc:208] All configs were filtered out because none of them sufficiently match the hints. Maybe the hints set does not contain a good representative set of valid configs? Working around this by using the full hints set instead.
2026-04-10 15:39:46.909958: I external/local_xla/xla/stream_executor/cuda/subprocess_compilation.cc:346] ptxas warning : Registers are spilled to local memory in function 'gemm_fusion_dot_979', 16 bytes spill stores, 16 bytes spill loads

2026-04-10 15:39:47.300555: I external/local_xla/xla/stream_executor/cuda/subprocess_compilation.cc:346] ptxas warning : Registers are spilled to local memory in function 'gemm_fusion_dot_1400', 16 bytes spill stores, 16 bytes spill loads

2026-04-10 15:39:47.393177: I external/local_xla/xla/stream_executor/cuda/subprocess_compilation.cc:346] ptxas warning : Registers are spilled to local memory in function 'gemm_fusion_dot_979', 4 bytes spill stores, 4 bytes spill loads

2026-04-10 15:39:47.615833: I external/local_xla/xla/stream_executor/cuda/subprocess_compilation.cc:346] ptxas warning : Registers are spilled to local memory in function 'gemm_fusion_dot_1400', 580 bytes spill stores, 580 bytes spill loads

2026-04-10 15:39:47.797040: I external/local_xla/xla/stream_executor/cuda/subprocess_compilation.cc:346] ptxas warning : Registers are spilled to local memory in function 'gemm_fusion_dot_1493', 100 bytes spill stores, 100 bytes spill loads

56/56  0s 51ms/step - loss: 7630.3670 - mae: 78.9934

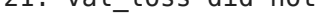
2026-04-10 15:39:49.399814: I external/local_xla/xla/service/gpu/autotuning/dot_search_space.cc:208] All configs were filtered out because none of them sufficiently match the hints. Maybe the hints set does not contain a good representative set of valid configs? Working around this by using the full hints set instead.
2026-04-10 15:39:49.606119: I external/local_xla/xla/stream_executor/cuda/subprocess_compilation.cc:346] ptxas warning : Registers are spilled to local memory in function 'gemm_fusion_dot_54', 4 bytes spill stores, 4 bytes spill loads

2026-04-10 15:39:49.884941: I external/local_xla/xla/stream_executor/cuda/subprocess_compilation.cc:346] ptxas warning : Registers are spilled to local memory in function 'gemm_fusion_dot_54', 16 bytes spill stores, 16 bytes spill loads

```

Epoch 1: val_loss improved from None to 9154.12305, saving model to ../CMA PSSData/best_model.keras
56/56  **7s** 71ms/step - loss: 7554.0278 - mae: 79.4247 - val_loss: 9154.1230 - val_mae: 88.1478 - learning_rate: 0.0010
Epoch 2/100
37/56  **0s** 1ms/step - loss: 7285.5121 - mae: 79.3370
Epoch 2: val_loss improved from 9154.12305 to 8561.82129, saving model to ../CMA PSSData/best_model.keras
56/56  **0s** 3ms/step - loss: 7017.1870 - mae: 78.3696 - val_loss: 8561.8213 - val_mae: 86.7810 - learning_rate: 0.0010
Epoch 3/100
40/56  **0s** 1ms/step - loss: 6657.8716 - mae: 77.4928
Epoch 3: val_loss improved from 8561.82129 to 8431.60352, saving model to ../CMA PSSData/best_model.keras
56/56  **0s** 3ms/step - loss: 6460.7612 - mae: 76.6645 - val_loss: 8431.6035 - val_mae: 87.7533 - learning_rate: 0.0010
Epoch 4/100
46/56  **0s** 1ms/step - loss: 6075.2657 - mae: 75.1593
Epoch 4: val_loss did not improve from 8431.60352
56/56  **0s** 2ms/step - loss: 5902.6167 - mae: 74.3191 - val_loss: 9083.2109 - val_mae: 92.5568 - learning_rate: 0.0010
Epoch 5/100
47/56  **0s** 1ms/step - loss: 5505.0685 - mae: 72.2658
Epoch 5: val_loss improved from 8431.60352 to 8050.12500, saving model to ../CMA PSSData/best_model.keras
56/56  **0s** 3ms/step - loss: 5355.0103 - mae: 71.3955 - val_loss: 8050.1250 - val_mae: 87.7799 - learning_rate: 0.0010
Epoch 6/100
45/56  **0s** 1ms/step - loss: 4907.1095 - mae: 68.4896
Epoch 6: val_loss did not improve from 8050.12500
56/56  **0s** 2ms/step - loss: 4841.7686 - mae: 68.0638 - val_loss: 8205.5391 - val_mae: 88.9111 - learning_rate: 0.0010
Epoch 7/100
46/56  **0s** 1ms/step - loss: 4443.6704 - mae: 65.1205
Epoch 7: val_loss improved from 8050.12500 to 6866.46143, saving model to ../CMA PSSData/best_model.keras
56/56  **0s** 3ms/step - loss: 4334.9751 - mae: 64.2747 - val_loss: 6866.4614 - val_mae: 81.3887 - learning_rate: 0.0010
Epoch 8/100
44/56  **0s** 1ms/step - loss: 3990.1307 - mae: 61.4595
Epoch 8: val_loss improved from 6866.46143 to 4935.90186, saving model to ../CMA PSSData/best_model.keras
56/56  **0s** 3ms/step - loss: 3833.2756 - mae: 60.1235 - val_loss: 4935.9019 - val_mae: 68.6230 - learning_rate: 0.0010
Epoch 9/100
40/56  **0s** 1ms/step - loss: 3519.7637 - mae: 57.4383
Epoch 9: val_loss improved from 4935.90186 to 4462.74414, saving model to ../CMA PSSData/best_model.keras
56/56  **0s** 3ms/step - loss: 3286.3438 - mae: 55.4464 - val_loss: 4462.7441 - val_mae: 65.0682 - learning_rate: 0.0010
Epoch 10/100
49/56  **0s** 1ms/step - loss: 2929.8099 - mae: 52.1859
Epoch 10: val_loss improved from 4462.74414 to 3381.77563, saving model to ../CMA PSSData/best_model.keras
56/56  **0s** 3ms/step - loss: 2726.9878 - mae: 50.1557 - val_loss: 3381.7756 - val_mae: 56.2951 - learning_rate: 0.0010
Epoch 11/100
48/56  **0s** 1ms/step - loss: 2381.2041 - mae: 46.6440
Epoch 11: val_loss improved from 3381.77563 to 3203.89087, saving model to ../CMA PSSData/best_model.keras

```

56/56  0s 3ms/step - loss: 2183.9995 - mae: 44.4519 -
val_loss: 3203.8909 - val_mae: 54.5336 - learning_rate: 0.0010
Epoch 12/100
43/56  0s 1ms/step - loss: 1802.0715 - mae: 40.0239
Epoch 12: val_loss improved from 3203.89087 to 2301.22607, saving model to
../CMAPSSData/best_model.keras
56/56  0s 3ms/step - loss: 1702.6373 - mae: 38.6980 -
val_loss: 2301.2261 - val_mae: 45.5971 - learning_rate: 0.0010
Epoch 13/100
44/56  0s 1ms/step - loss: 1339.2441 - mae: 33.8665
Epoch 13: val_loss improved from 2301.22607 to 1342.38696, saving model to
../CMAPSSData/best_model.keras
56/56  0s 3ms/step - loss: 1286.2092 - mae: 33.0246 -
val_loss: 1342.3870 - val_mae: 33.8057 - learning_rate: 0.0010
Epoch 14/100
44/56  0s 1ms/step - loss: 1010.3473 - mae: 28.6342
Epoch 14: val_loss did not improve from 1342.38696
56/56  0s 2ms/step - loss: 952.9750 - mae: 27.7525 - v
al_loss: 1381.4186 - val_mae: 34.0460 - learning_rate: 0.0010
Epoch 15/100
46/56  0s 1ms/step - loss: 772.8023 - mae: 24.4648
Epoch 15: val_loss improved from 1342.38696 to 973.50153, saving model to
../CMAPSSData/best_model.keras
56/56  0s 3ms/step - loss: 697.7158 - mae: 23.0147 - v
al_loss: 973.5015 - val_mae: 27.4915 - learning_rate: 0.0010
Epoch 16/100
48/56  0s 1ms/step - loss: 555.1180 - mae: 20.0438
Epoch 16: val_loss improved from 973.50153 to 835.47705, saving model to
../CMAPSSData/best_model.keras
56/56  0s 3ms/step - loss: 522.1060 - mae: 19.3379 - v
al_loss: 835.4771 - val_mae: 25.1510 - learning_rate: 0.0010
Epoch 17/100
46/56  0s 1ms/step - loss: 403.1403 - mae: 16.5595
Epoch 17: val_loss improved from 835.47705 to 534.26385, saving model to
../CMAPSSData/best_model.keras
56/56  0s 3ms/step - loss: 400.7492 - mae: 16.5095 - v
al_loss: 534.2639 - val_mae: 19.7216 - learning_rate: 0.0010
Epoch 18/100
46/56  0s 1ms/step - loss: 312.9674 - mae: 14.2347
Epoch 18: val_loss improved from 534.26385 to 447.48352, saving model to
../CMAPSSData/best_model.keras
56/56  0s 3ms/step - loss: 317.5052 - mae: 14.3669 - v
al_loss: 447.4835 - val_mae: 17.4571 - learning_rate: 0.0010
Epoch 19/100
26/56  0s 2ms/step - loss: 293.8733 - mae: 13.6396
Epoch 19: val_loss improved from 447.48352 to 269.04254, saving model to
../CMAPSSData/best_model.keras
56/56  0s 3ms/step - loss: 273.9262 - mae: 13.1965 - v
al_loss: 269.0425 - val_mae: 13.1570 - learning_rate: 0.0010
Epoch 20/100
45/56  0s 1ms/step - loss: 253.9416 - mae: 12.6904
Epoch 20: val_loss did not improve from 269.04254
56/56  0s 2ms/step - loss: 250.0405 - mae: 12.4964 - v
al_loss: 329.3732 - val_mae: 14.3092 - learning_rate: 0.0010
Epoch 21/100
44/56  0s 1ms/step - loss: 232.9664 - mae: 11.9405
Epoch 21: val_loss did not improve from 269.04254
56/56  0s 2ms/step - loss: 235.6810 - mae: 12.0233 - v
al_loss: 315.8067 - val_mae: 14.0108 - learning_rate: 0.0010
Epoch 22/100

```

```

45/56  0s 1ms/step - loss: 217.9463 - mae: 11.6016
Epoch 22: val_loss improved from 269.04254 to 201.63477, saving model to
../CMAPSSData/best_model.keras
56/56  0s 3ms/step - loss: 225.2265 - mae: 11.7257 - v
al_loss: 201.6348 - val_mae: 10.8747 - learning_rate: 0.0010
Epoch 23/100
51/56  0s 1ms/step - loss: 216.6739 - mae: 11.4967
Epoch 23: val_loss did not improve from 201.63477
56/56  0s 2ms/step - loss: 218.4855 - mae: 11.5304 - v
al_loss: 219.6082 - val_mae: 11.2358 - learning_rate: 0.0010
Epoch 24/100
41/56  0s 1ms/step - loss: 212.0068 - mae: 11.3538
Epoch 24: val_loss did not improve from 201.63477
56/56  0s 2ms/step - loss: 213.9885 - mae: 11.3743 - v
al_loss: 213.8237 - val_mae: 11.2721 - learning_rate: 0.0010
Epoch 25/100
46/56  0s 1ms/step - loss: 219.6085 - mae: 11.5771
Epoch 25: val_loss did not improve from 201.63477
56/56  0s 2ms/step - loss: 216.2098 - mae: 11.4760 - v
al_loss: 203.9267 - val_mae: 10.6866 - learning_rate: 0.0010
Epoch 26/100
45/56  0s 1ms/step - loss: 210.8301 - mae: 11.3161
Epoch 26: val_loss did not improve from 201.63477
56/56  0s 2ms/step - loss: 210.6019 - mae: 11.2811 - v
al_loss: 216.1110 - val_mae: 11.7379 - learning_rate: 0.0010
Epoch 27/100
44/56  0s 1ms/step - loss: 210.8753 - mae: 11.3604
Epoch 27: val_loss improved from 201.63477 to 190.73860, saving model to
../CMAPSSData/best_model.keras
56/56  0s 3ms/step - loss: 208.4859 - mae: 11.2382 - v
al_loss: 190.7386 - val_mae: 10.7187 - learning_rate: 0.0010
Epoch 28/100
46/56  0s 1ms/step - loss: 204.2202 - mae: 11.0948
Epoch 28: val_loss did not improve from 190.73860
56/56  0s 2ms/step - loss: 209.0642 - mae: 11.2537 - v
al_loss: 250.7032 - val_mae: 12.4362 - learning_rate: 0.0010
Epoch 29/100
49/56  0s 1ms/step - loss: 201.2929 - mae: 10.9617
Epoch 29: val_loss did not improve from 190.73860
56/56  0s 2ms/step - loss: 201.8439 - mae: 11.0053 - v
al_loss: 261.9574 - val_mae: 12.8480 - learning_rate: 0.0010
Epoch 30/100
48/56  0s 1ms/step - loss: 200.4176 - mae: 10.9829
Epoch 30: val_loss did not improve from 190.73860
56/56  0s 2ms/step - loss: 198.3033 - mae: 10.9402 - v
al_loss: 213.5125 - val_mae: 11.4727 - learning_rate: 0.0010
Epoch 31/100
46/56  0s 1ms/step - loss: 195.2939 - mae: 10.8474
Epoch 31: val_loss did not improve from 190.73860
56/56  0s 2ms/step - loss: 199.8002 - mae: 11.0017 - v
al_loss: 195.1630 - val_mae: 10.7334 - learning_rate: 0.0010
Epoch 32/100
44/56  0s 1ms/step - loss: 193.7321 - mae: 10.7892
Epoch 32: ReduceLR0nPlateau reducing learning rate to 0.000500000023748725
7.

Epoch 32: val_loss did not improve from 190.73860
56/56  0s 2ms/step - loss: 196.3134 - mae: 10.8444 - v
al_loss: 192.2299 - val_mae: 10.4368 - learning_rate: 0.0010
Epoch 33/100

```



```

27/56  0s 2ms/step - loss: 189.5240 - mae: 10.6865
Epoch 33: val_loss did not improve from 190.73860
56/56  0s 3ms/step - loss: 194.9768 - mae: 10.8040 - v
al_loss: 215.1030 - val_mae: 11.4577 - learning_rate: 5.0000e-04
Epoch 34/100
43/56  0s 1ms/step - loss: 184.6371 - mae: 10.4613
Epoch 34: val_loss improved from 190.73860 to 177.73224, saving model to
../CMAPSSData/best_model.keras
56/56  0s 3ms/step - loss: 191.1053 - mae: 10.6435 - v
al_loss: 177.7322 - val_mae: 9.9950 - learning_rate: 5.0000e-04
Epoch 35/100
46/56  0s 1ms/step - loss: 189.0569 - mae: 10.6390
Epoch 35: val_loss did not improve from 177.73224
56/56  0s 2ms/step - loss: 189.3226 - mae: 10.6768 - v
al_loss: 178.3438 - val_mae: 9.8578 - learning_rate: 5.0000e-04
Epoch 36/100
48/56  0s 1ms/step - loss: 188.8650 - mae: 10.6913
Epoch 36: val_loss did not improve from 177.73224
56/56  0s 2ms/step - loss: 189.9418 - mae: 10.7061 - v
al_loss: 184.7506 - val_mae: 10.2501 - learning_rate: 5.0000e-04
Epoch 37/100
43/56  0s 1ms/step - loss: 189.6349 - mae: 10.6716
Epoch 37: val_loss did not improve from 177.73224
56/56  0s 2ms/step - loss: 189.0642 - mae: 10.6538 - v
al_loss: 230.3195 - val_mae: 11.5809 - learning_rate: 5.0000e-04
Epoch 38/100
46/56  0s 1ms/step - loss: 182.0652 - mae: 10.5292
Epoch 38: val_loss did not improve from 177.73224
56/56  0s 2ms/step - loss: 185.5692 - mae: 10.6180 - v
al_loss: 282.3042 - val_mae: 12.5217 - learning_rate: 5.0000e-04
Epoch 39/100
43/56  0s 1ms/step - loss: 186.3118 - mae: 10.5675
Epoch 39: ReduceLRonPlateau reducing learning rate to 0.000250000011874362
8.

Epoch 39: val_loss did not improve from 177.73224
56/56  0s 2ms/step - loss: 190.1510 - mae: 10.7078 - v
al_loss: 201.0257 - val_mae: 10.8265 - learning_rate: 5.0000e-04
Epoch 40/100
44/56  0s 1ms/step - loss: 189.4300 - mae: 10.6955
Epoch 40: val_loss did not improve from 177.73224
56/56  0s 2ms/step - loss: 190.7521 - mae: 10.7289 - v
al_loss: 206.8039 - val_mae: 10.7486 - learning_rate: 2.5000e-04
Epoch 41/100
46/56  0s 1ms/step - loss: 191.9946 - mae: 10.7147
Epoch 41: val_loss did not improve from 177.73224
56/56  0s 2ms/step - loss: 190.5132 - mae: 10.7036 - v
al_loss: 183.4013 - val_mae: 10.0860 - learning_rate: 2.5000e-04
Epoch 42/100
48/56  0s 1ms/step - loss: 183.2929 - mae: 10.4797
Epoch 42: val_loss improved from 177.73224 to 177.28841, saving model to
../CMAPSSData/best_model.keras
56/56  0s 3ms/step - loss: 185.2316 - mae: 10.5479 - v
al_loss: 177.2884 - val_mae: 10.0403 - learning_rate: 2.5000e-04
Epoch 43/100
48/56  0s 1ms/step - loss: 182.6993 - mae: 10.4951
Epoch 43: val_loss improved from 177.28841 to 175.76208, saving model to
../CMAPSSData/best_model.keras
56/56  0s 3ms/step - loss: 187.0922 - mae: 10.6023 - v
al_loss: 175.7621 - val_mae: 9.9185 - learning_rate: 2.5000e-04

```

```

Epoch 44/100
45/56  0s 1ms/step - loss: 184.4661 - mae: 10.4433
Epoch 44: val_loss did not improve from 175.76208
56/56  0s 2ms/step - loss: 182.8868 - mae: 10.4526 - v
al_loss: 182.2755 - val_mae: 9.9790 - learning_rate: 2.5000e-04
Epoch 45/100
23/56  0s 2ms/step - loss: 192.8999 - mae: 10.8236
Epoch 45: val_loss did not improve from 175.76208
56/56  0s 3ms/step - loss: 189.2959 - mae: 10.6589 - v
al_loss: 182.4565 - val_mae: 10.1351 - learning_rate: 2.5000e-04
Epoch 46/100
44/56  0s 1ms/step - loss: 190.0226 - mae: 10.7061
Epoch 46: val_loss improved from 175.76208 to 172.65050, saving model to
../CMAPSSData/best_model.keras
56/56  0s 3ms/step - loss: 188.4116 - mae: 10.6245 - v
al_loss: 172.6505 - val_mae: 9.7729 - learning_rate: 2.5000e-04
Epoch 47/100
44/56  0s 1ms/step - loss: 183.8392 - mae: 10.5071
Epoch 47: val_loss did not improve from 172.65050
56/56  0s 2ms/step - loss: 184.1951 - mae: 10.5057 - v
al_loss: 201.9129 - val_mae: 10.7586 - learning_rate: 2.5000e-04
Epoch 48/100
49/56  0s 1ms/step - loss: 184.9851 - mae: 10.5382
Epoch 48: val_loss did not improve from 172.65050
56/56  0s 2ms/step - loss: 182.3452 - mae: 10.4815 - v
al_loss: 184.0991 - val_mae: 10.1449 - learning_rate: 2.5000e-04
Epoch 49/100
47/56  0s 1ms/step - loss: 183.6610 - mae: 10.4862
Epoch 49: val_loss did not improve from 172.65050
56/56  0s 2ms/step - loss: 190.6007 - mae: 10.7307 - v
al_loss: 177.3538 - val_mae: 9.8825 - learning_rate: 2.5000e-04
Epoch 50/100
47/56  0s 1ms/step - loss: 179.8675 - mae: 10.3802
Epoch 50: val_loss did not improve from 172.65050
56/56  0s 2ms/step - loss: 184.6025 - mae: 10.5019 - v
al_loss: 175.9569 - val_mae: 9.9348 - learning_rate: 2.5000e-04
Epoch 51/100
45/56  0s 1ms/step - loss: 186.3179 - mae: 10.6106
Epoch 51: ReduceLRonPlateau reducing learning rate to 0.000125000005937181
4.

Epoch 51: val_loss did not improve from 172.65050
56/56  0s 2ms/step - loss: 181.8437 - mae: 10.4389 - v
al_loss: 180.3668 - val_mae: 10.0019 - learning_rate: 2.5000e-04
Epoch 52/100
45/56  0s 1ms/step - loss: 179.6613 - mae: 10.3540
Epoch 52: val_loss did not improve from 172.65050
56/56  0s 2ms/step - loss: 180.3679 - mae: 10.3847 - v
al_loss: 179.5525 - val_mae: 9.9259 - learning_rate: 1.2500e-04
Epoch 53/100
40/56  0s 1ms/step - loss: 187.9648 - mae: 10.5786
Epoch 53: val_loss did not improve from 172.65050
56/56  0s 2ms/step - loss: 182.0109 - mae: 10.4432 - v
al_loss: 178.1456 - val_mae: 9.8877 - learning_rate: 1.2500e-04
Epoch 54/100
48/56  0s 1ms/step - loss: 183.9881 - mae: 10.5722
Epoch 54: val_loss did not improve from 172.65050
56/56  0s 2ms/step - loss: 182.5961 - mae: 10.4752 - v
al_loss: 176.1937 - val_mae: 9.8260 - learning_rate: 1.2500e-04
Epoch 55/100

```


46/56  0s 1ms/step - loss: 178.7846 - mae: 10.3652
 Epoch 55: val_loss did not improve from 172.65050
 56/56  0s 2ms/step - loss: 181.0056 - mae: 10.4347 - val_loss: 176.3053 - val_mae: 9.8233 - learning_rate: 1.2500e-04
 Epoch 56/100
 49/56  0s 1ms/step - loss: 178.7307 - mae: 10.3660
 Epoch 56: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.
 Epoch 56: val_loss did not improve from 172.65050
 56/56  0s 2ms/step - loss: 182.2547 - mae: 10.4584 - val_loss: 176.9723 - val_mae: 9.8695 - learning_rate: 1.2500e-04
 Epoch 56: early stopping
 Restoring model weights from the end of the best epoch: 46.

```
In [20]: import matplotlib.pyplot as plt

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 4))

ax1.plot(history.history['loss'], label='Train Loss')
ax1.plot(history.history['val_loss'], label='Val Loss')
ax1.set_title('MSE Loss')
ax1.set_xlabel('Epoch')
ax1.legend()

ax2.plot(history.history['mae'], label='Train MAE')
ax2.plot(history.history['val_mae'], label='Val MAE')
ax2.set_title('MAE')
ax2.set_xlabel('Epoch')
ax2.legend()

plt.suptitle('Training History - FD001', fontsize=13)
plt.tight_layout()
plt.savefig('C-MAPSS/CMAPSSData/processed/training_history.png', dpi=150)
plt.show()
```



```
In [21]: from sklearn.metrics import mean_squared_error

# Predict
y_pred = model.predict(X_test).flatten()

# Clip predictions to valid range
y_pred = np.clip(y_pred, 0, 125)

# Metrics
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = np.mean(np.abs(y_pred - y_test))

print("=" * 40)
```

```

print(f"  RMSE : {rmse:.4f}")
print(f"  MAE  : {mae:.4f}")
print("=" * 40)

# Visualize predicted vs actual
plt.figure(figsize=(12, 4))
plt.plot(y_test[:300], label='Actual RUL', alpha=0.8)
plt.plot(y_pred[:300], label='Predicted RUL', alpha=0.8)
plt.title('Predicted vs Actual RUL – First 300 Test Samples')
plt.xlabel('Sample')
plt.ylabel('RUL')
plt.legend()
plt.tight_layout()
plt.savefig('C-MAPSS/CMAPSSData/processed/rul_predictions.png', dpi=150)
plt.show()

```

2026-04-10 15:41:10.368288: I external/local_xla/xla/service/gpu/autotuning/dot_search_space.cc:208] All configs were filtered out because none of them sufficiently match the hints. Maybe the hints set does not contain a good representative set of valid configs? Working around this by using the full hints set instead.

2026-04-10 15:41:10.899904: I external/local_xla/xla/stream_executor/cuda/subprocess_compilation.cc:346] ptxas warning : Registers are spilled to local memory in function 'gemm_fusion_dot_42', 24 bytes spill stores, 24 bytes spill loads

240/319 ————— **0s** 63lus/step

2026-04-10 15:41:11.419990: I external/local_xla/xla/service/gpu/autotuning/dot_search_space.cc:208] All configs were filtered out because none of them sufficiently match the hints. Maybe the hints set does not contain a good representative set of valid configs? Working around this by using the full hints set instead.

2026-04-10 15:41:11.898096: I external/local_xla/xla/stream_executor/cuda/subprocess_compilation.cc:346] ptxas warning : Registers are spilled to local memory in function 'gemm_fusion_dot_42', 28 bytes spill stores, 28 bytes spill loads

2026-04-10 15:41:11.973854: I external/local_xla/xla/stream_executor/cuda/subprocess_compilation.cc:346] ptxas warning : Registers are spilled to local memory in function 'gemm_fusion_dot_42', 24 bytes spill stores, 24 bytes spill loads

319/319 ————— **2s** 4ms/step

=====

RMSE : 13.2492

MAE : 9.5331

=====

